

GVCCTurbo: Multi-Resolution Coding and Predictive Skip for Generative Video Compression

Anonymous WACV Algorithms Track submission

Paper ID *****

Abstract

Generative video codecs that drive a pretrained rectified-flow generator with a transmitted noise codebook recently achieved competitive perceptual quality at ultra-low bitrates, but their decode-time cost—tens to hundreds of generator forward passes per Group of Pictures (GOP)—is one to two orders of magnitude above the operating point of practical learned video codecs. We present a turbo-charged variant of this design (GVCCTurbo) that retains the underlying zero-shot codebook scheme but accelerates per-GOP decoding by up to $1.9\times$ on the published Wan 2.1 baselines, with quality essentially preserved at a small bitrate increase.

Two complementary mechanisms drive the speedup. First, a multi-resolution coding schedule runs the early high-noise iterations at a fraction of the full spatial resolution and only switches to the target resolution near the clean end. We identify and fix a previously overlooked target-domain mismatch between low- and high-resolution stages: encoding the low stage on the spatially downsampled video is not equivalent to taking the DCT-low projection of the high-resolution latent, and naively chaining the two imposes a structural ceiling of approximately 1.4 dB. Unifying both stages in the high-resolution latent domain via DCT projection collapses this gap to 0.2 dB with only a small bitrate change. Second, a predictive-skip mechanism caches the generator’s clean-latent estimate rather than its velocity, allowing every other generator call along the trajectory to be reconstructed from the cached endpoint. Combined with a one-step warmup after each resolution transition, this yields an overall $1.7\text{--}1.9\times$ decoder speedup with a small quality cost (< 0.3 dB PSNR in our main setting). Both fixes are obtained from a trajectory-geometric reading of the generator’s flow, with negligible engineering overhead.

We evaluate GVCCTurbo on UVG at 720p and

1080p across three GOP-conditioning modes (T2V, I2V, FLF2V). At 1080p with first-last-frame conditioning, the turbo pipeline matches the published GVCC operating point in bitrate, improves reconstruction metrics by up to +0.5 dB PSNR and -14% LPIPS, and reduces per-GOP decode time by approximately $2\times$. Code, per-GOP measurements, and codebook bitstreams will be released to support reproducibility.

1. Introduction

Practical video compression has long been dominated by spatial-temporal hybrid codecs in the HEVC/VVC tradition, in which intra-coded keyframes anchor a sequence of motion-compensated predictions and small per-block residuals are entropy-coded. Learned video codecs of the DCVC family [5, 7] have closed and in many cases overtaken the rate-distortion (RD) frontier of these traditional schemes while preserving the same Group-of-Pictures (GOP) intuition: a small, strongly conditioned model produces a deterministic reconstruction, and the transmitted bits describe how much the actual frames differ from that reconstruction.

A complementary line of work has recently shown that pretrained generative video models can themselves serve as the decoder of an ultra-low-bitrate codec. In this setting, the encoder transmits not a residual against a deterministic prediction, but a compact specification of which sample of the model’s stochastic generation process to reproduce. GVCC [23] instantiates this idea by converting a rectified-flow video generator into a marginal-preserving stochastic process, and replacing the per-step Gaussian noise of that process with codebook-indexed innovations that the encoder selects to match the ground-truth video. The result is a zero-shot codec—no training of the generator or any auxiliary network—that on UVG achieves substantially lower LPIPS than HEVC, VVC, and even strong learned codecs at sub-0.01 bpp.

075	The clear weakness of this design is decoder time.	126
076	Each GOP requires $T \approx 20$ forward passes through	127
077	a multi-billion-parameter video generator, plus a per-	
078	step codebook search; on the published Wan 2.1-	
079	14B [20] backbone, a single 33-frame 1080p GOP takes	
080	several hundred seconds to decode on a high-end GPU.	
081	This is one to two orders of magnitude above the op-	
082	erating point of DCVC-RT and rules out the codec in	
083	any latency-sensitive scenario.	
084	In this paper we present GVCCTurbo, a turbo-	
085	charged variant of the same generative-codec design	
086	that preserves the zero-shot codebook scheme but ac-	
087	celerates per-GOP decoding by up to $1.9\times$ on Wan 2.1-	
088	14B without measurable RD degradation. Our two	
089	contributions are simple, trajectory-geometric, and re-	
090	quire no retraining.	
091	Multi-resolution coding with a unified target. A nat-	
092	ural way to cut decoder cost is to run the early high-	
093	noise iterations of the generator at a lower spatial res-	
094	olution and only switch to the target resolution near	
095	the clean end—a video-coding analogue of progressive	
096	spectral diffusion [21]. We find that the obvious instan-	
097	tiation (encode at low-res VAE, switch to high-res VAE	
098	via a spectral lift) hits a structural ceiling of roughly	
099	1.4 dB on UVG that persists across backbone scale and	
100	stage count, and that does not close under a joint or-	
101	acle. The underlying cause is that the low-resolution	
102	VAE latent and the DCT-low projection of the high-	
103	resolution VAE latent are different objects, not aligned	
104	by the spectral lift used at transition. Unifying both	
105	stages in the high-resolution latent domain—using the	
106	DCT-low projection of the high-res latent as the low-	
107	stage codebook target—collapses the ceiling to 0.2 dB	
108	at the same bitrate, recovering nearly all of the gap	
109	with a one- line change in the codec target.	
110	Predictive skip via clean-endpoint caching. The dom-	
111	inant per-step cost is the video generator’s velocity	
112	evaluation. We replace half of these evaluations along	
113	the SDE trajectory with a cached reconstruction. The	
114	naive way to cache—storing the previous step’s velocity	
115	and reusing it—silently drifts, because the velocity is a	
116	function of both state and time. Instead, we cache the	
117	generator’s clean-image estimate (the trajectory end-	
118	point) and reconstruct the skipped step’s velocity from	
119	the current state and current time. Under the linear	
120	flow interpolant, the clean endpoint is invariant along	
121	the trajectory, so this scheme is geometrically faith-	
122	ful within a local linearization. We show empirically	
123	that the cache has an effective radius of one step: a	
124	fixed alternating skip/run schedule, combined with a	
125	one-step warmup after each resolution transition, is	
	Pareto-optimal under this caching scheme; more ag-	
	gressive multi-step skipping is unstable.	
	GOP-level evaluation across conditioning modes.	128
	GVCC is evaluated in three GOP-conditioning modes:	129
	text-only (T2V), with a single anchor frame (I2V) with	130
	autoregressive chaining, and with both endpoint frames	131
	(FLF2V) with boundary sharing across consecutive	132
	GOPs. We retain this three-way structure and ap-	133
	ply our acceleration mechanisms across all three modes	134
	on UVG at 720p and 1080p. At 1080p with first-last-	135
	frame conditioning, the turbo pipeline matches the bi-	136
	trate of the published GVCC operating point and ex-	137
	ceeds it by +0.5 dB PSNR and -14% LPIPS, while	138
	reducing per-GOP decode time by approximately $2\times$.	139
	The remainder of the paper is organized as fol-	140
	lows. Section 2 positions our work against traditional,	141
	learned, and generative video codecs. Section 3 de-	142
	scribes the pipeline and derives both acceleration mech-	143
	anisms, including the trajectory-geometric arguments	144
	and falsified alternatives. Section 4 reports the main	145
	RD and decode-time comparison on UVG. Section 5	146
	isolates the contribution of each component and val-	147
	idates the one-step-radius result for predictive skip-	148
	ping.	149
	2. Related Work	150
	Traditional and learned video codecs. Standardized	151
	hybrid codecs HEVC [18] and VVC [2] structure a se-	152
	quence into intra (I), predicted (P), and bidirectional	153
	(B) frames, transmitting residuals against motion-	154
	compensated predictions through DCT and entropy	155
	coding. Learned video codecs largely retain this GOP	156
	layout but replace each component with neural mod-	157
	ules: DCVC and its descendants [5, 7] use learned con-	158
	ditional entropy models and feature-space prediction.	159
	These methods now match or exceed VVC in PSNR at	160
	the standard 0.05–0.20 bpp operating range with real-	161
	time-class decode speeds, but their RD curves rapidly	162
	flatten below 0.01 bpp.	163
	Generative video compression. A separate line of	164
	work uses generative models to sample a plausible re-	165
	construction at ultra-low bitrates rather than regress	166
	to a deterministic prediction. GLC-Video [15] and	167
	GNVC-VD [12] train end-to-end generative codecs that	168
	achieve 0.10–0.15 LPIPS at 0.003 bpp, well below the	169
	LPIPS of traditional and learned codecs at the same	170
	rate, but require dedicated training. Free-GVC [9]	171
	avoids training by performing reverse channel coding	172
	along the trajectory of a pretrained CogVideoX gener-	173
	ator. GVCC [23] is the closest predecessor to our work:	174

it converts a pretrained rectified-flow video generator into a stochastic process and transmits codebook-indexed per-step noise. We inherit GVCC’s codec backbone—including its Turbo-DDCM [19] multi-atom codebook construction—and focus exclusively on closing the decode-time gap left open by that work.

Multi-resolution diffusion sampling. The idea of running the early high-noise iterations of a diffusion or flow model at lower resolution is well established in the sampling literature. Spectral Progressive Diffusion (SPD) [21] provides a clean spectral-domain reading: at each timestep t , only those frequency bands whose signal-to-noise ratio crosses a threshold need to be active, and lower bands cross first. Cascaded diffusion [4] and several text-to-image systems [3, 14] exploit the same structure with multiple separately trained models.

Our use of multi-resolution coding differs from these in two ways. First, the goal is not to generate plausible samples but to drive an encoder-specified target with bit-exact reproducibility on both sides. Second, the low and high stages share a single pretrained backbone (no cascading of distinct models), which forces a precise specification of what the low stage’s target latent is—a question that does not arise in generation, where any plausible low-frequency content is acceptable. The target-domain mismatch we identify in Section 3.2 arises exactly because the low stage is anchored by the encoder, not free to drift.

Accelerating diffusion sampling. Reducing the number of model evaluations is well studied in the distillation literature: progressive distillation [16], consistency models [10, 17], and Sana [22]-style mobile distillations all aim to bring sample counts from $T = 20$ –50 down to $T \leq 4$. These approaches require retraining and produce a distilled generator that may not be compatible with the encoder’s per-step codebook reproducibility requirement. Closer to us are inference-time caching schemes that reuse intermediate quantities across denoising steps: DeepCache [11] caches internal U-Net features, and several recent works [6] cache velocity outputs directly. We adopt the latter family but cache the clean-image endpoint rather than the velocity (Section 3.3); this choice is forced by the codec requirement that the cache stay valid across an SDE step where the trajectory state changes by an encoder-selected codebook innovation.

Pretrained video generators as the decoder backbone. We instantiate GVCCTurbo on Wan 2.1 [20], a rectified-flow video generator supporting unconditional

(T2V), single-anchor (I2V), and dual-anchor (FLF2V) generation modes at 480p and 720p+. The choice follows [23] for direct comparability; the framework inherits whatever resolution and conditioning support the base generator provides.

3. Method

We treat each F -frame GOP as the unit of compression. The decoder reconstructs the GOP by running a pretrained rectified-flow video generator forward for T discrete steps; the encoder selects per-step codebook indices so that the resulting trajectory matches the ground-truth GOP. Section 3.1 states this codec backbone in codec-centric terms (inherited from [19, 23] in compact form), and the remaining subsections describe the two acceleration mechanisms.

3.1. Codec backbone

Encoder/decoder pipeline. A pretrained 3D VAE maps a GOP $\mathbf{V} \in \mathbb{R}^{F \times H \times W \times 3}$ to a latent $\mathbf{x}_0 \in \mathbb{R}^{F_l \times H_l \times W_l \times C}$. The decoder is a rectified-flow video generator whose ODE $\dot{\mathbf{x}}_t = \mathbf{u}_\theta(\mathbf{x}_t, t)$ transports samples from $t=1$ (pure noise) to $t=0$ (clean latent). We discretize this trajectory at T timesteps $\{t_k\}_{k=1}^T$ and inject stochastic innovations at the first T_{sde} of them, leaving an $N = T - T_{\text{sde}}$ -step deterministic ODE tail:

$$\mathbf{x}_{t_{k+1}} = \mathbf{x}_{t_k} - f_{t_k} \Delta t_k + g_{t_k} \sqrt{\Delta t_k} \mathbf{z}_k^*, \quad (1)$$

where f_{t_k}, g_{t_k} are the SDE drift and diffusion coefficients derived from $\mathbf{u}_\theta$ as in [23], and $\mathbf{z}_k^*$ is the per-step innovation that the encoder transmits via a $\log_2(K)$ -bit codebook index together with a sign bit, per latent frame. The full bitstream therefore consists of $T_{\text{sde}} \cdot F_l \cdot M \cdot (\log_2 K + 1)$ codebook bits per GOP, plus any boundary-frame bits required by the conditioning mode (Section 3.4).

Codebook construction. We use the multi-atom construction of Turbo-DDCM [19]: at step k and latent frame f , the encoder generates K i.i.d. Gaussian atoms with a deterministic seed shared by both sides, computes inner products with the per-frame residual $\mathbf{r}_{k,f} = \mathbf{x}_{0,f} - \hat{\mathbf{x}}_{0|t_k,f}$, and transmits the M atoms with the largest absolute inner product together with their signs. The decoder regenerates the same K atoms, indexes into them, and reconstructs $\mathbf{z}_{k,f}^*$ as a unit-variance sum.

Cost. A single decode requires T video-generator evaluations and T_{sde} codebook reconstructions. At $T = 20$, $N = 3$, $K = 16384$, $M = 80$, the per-GOP decoder cost on a 14B-parameter Wan 2.1 model at 1080p

is approximately 700 s on an RTX PRO 6000. The two acceleration mechanisms below target the dominant cost—the per-step generator evaluation—without changing the codec backbone.

3.2. Multi-resolution coding with a unified target

Resolution schedule. We replace the single-resolution trajectory with an S -stage progressive schedule, $s = 0, \dots, S-1$, where stage s operates on a latent of spatial size $(H_l^{(s)}, W_l^{(s)})$ with $H_l^{(0)} \leq \dots \leq H_l^{(S-1)} = H_l$. Stage s denoises for τ_s codebook steps; at transition step $\tau_{s \rightarrow s+1}$ the trajectory is lifted to the next resolution via a 2D DCT zero-pad in the new band:

$$\bar{\mathbf{x}}_{t_k}^{(s+1)} = \mathcal{T}^{-1}(\text{Embed}(\mathcal{T}(\mathbf{x}_{t_k}^{(s)})) + t_k \epsilon_H^{(s+1)}), \quad (2)$$

where $\mathcal{T}$ is the spatial DCT, Embed pads the new high-frequency band with zeros, and $\epsilon_H^{(s+1)}$ is a fresh Gaussian sample on that band. Equation 2 is spectrally correct: the low band passes through unchanged and the new band matches the marginal distribution expected by the generator at the new resolution and timestep t_k .

The target-domain mismatch. For the encoder to drive the low-stage trajectory, it needs a low-stage ground-truth latent $\mathbf{x}_0^{(s)}$. The natural choice is

$$\mathbf{x}_0^{(s), \text{naive}} = \text{VAE}^{(s)}(\text{down}(\mathbf{V})), \quad (3)$$

i.e., spatially downsample the pixel video and re-encode at the lower resolution. This is what a single-resolution codec would do at each scale independently. The decoder, however, does not produce $\mathbf{x}_0^{(s), \text{naive}}$ at the end of stage s : it produces the trajectory whose low band, after Eq. 2 lifts it, agrees with $\mathcal{T}_L(\text{VAE}(\mathbf{V}))$ —the DCT-low projection of the high-resolution latent. The two are different objects:

$$\text{VAE}^{(s)}(\text{down}(\mathbf{V})) \neq \mathcal{T}_L(\text{VAE}(\mathbf{V})), \quad (4)$$

because the VAE encoder is nonlinear and its receptive field on the downsampled pixel support is not the corresponding DCT projection of its receptive field on the full-resolution support. On UVG with the Wan 2.1 VAE we measure a relative gap $\|\text{lhs} - \text{rhs}\| / \|\text{rhs}\| \approx 0.41$ at $t = 0$ on the low band. The encoder under Eq. 3 drives the trajectory toward an object the decoder cannot produce; the high-band fill at transition cannot recover the mismatch.

Fix: unify both stages in the high-resolution latent domain. We replace Eq. 3 with

$$\mathbf{x}_0^{(s), \text{ours}} = \mathcal{T}_L(\text{VAE}(\mathbf{V})), \quad (5)$$

i.e., encode the pixel video once at full resolution and project to the low band by DCT truncation. Both stages now reference the same latent $\mathbf{x}_0 = \text{VAE}(\mathbf{V})$, observed at different spectral bandwidths. Transition (Eq. 2) stops being a domain change and becomes pure band expansion: stage s converges to $\mathcal{T}_L(\mathbf{x}_0)$, and stage $s+1$ adds $\mathcal{T}_H(\mathbf{x}_0)$, recovering the same $\mathbf{x}_0$ that the encoder is matching against. The bitstream is unchanged; only the encoder’s matching target moves to a spectrally consistent reference.

Transition codebook. At the transition step $\tau_{s \rightarrow s+1}$, the newly opened high band $\mathbf{x}_{t_k}^{H_{s+1}}$ is by default filled with pure noise $t_k \epsilon_H$ (Eq. 2). The encoder knows the ideal high-band content $(1 - t_k) \mathcal{T}_{H_{s+1}}(\mathbf{x}_0)$, so the trajectory-consistent residual is $\mathbf{r}_{H_{s+1}} = (1 - t_k) \mathcal{T}_{H_{s+1}}(\mathbf{x}_0)$. We transmit one additional codebook atom per latent frame at this step, selected by inner product with $\mathbf{r}_{H_{s+1}}$. This costs $< 1\%$ of the per-GOP bit budget but recovers up to 0.4 dB PSNR over random high-band fill (Section 5).

3.3. Predictive skip via clean-endpoint caching

The dominant per-step cost is the video-generator evaluation $\mathbf{u}_\theta(\mathbf{x}_t, t)$. We aim to replace approximately half of these evaluations along the codebook-driven trajectory.

Why velocity caching fails. The naive choice is to cache the velocity itself:

$$\mathbf{u}_{k+1}^{\text{naive}} = \mathbf{u}_\theta(\mathbf{x}_{t_k}, t_k), \quad (6)$$

i.e., reuse the previous step’s direction. This silently drifts: $\mathbf{u}_\theta$ is a function of both $\mathbf{x}$ and t , and the trajectory has moved away from $\mathbf{x}_{t_k}$ by both a deterministic drift and an encoder-injected codebook innovation in Eq. 1. The cached direction is computed at a stale state, not the current one.

The fix: cache the trajectory endpoint. Under the linear flow interpolant $\mathbf{x}_t = (1 - t) \mathbf{x}_0 + t \epsilon$, the generator’s MMSE estimate of $\mathbf{x}_0$ at any point on the trajectory is

$$\hat{\mathbf{x}}_{0|t_k} = \mathbf{x}_{t_k} - t_k \mathbf{u}_\theta(\mathbf{x}_{t_k}, t_k). \quad (7)$$

$\hat{\mathbf{x}}_{0|t_k}$ is, geometrically, where the model thinks the trajectory ends. We cache this endpoint instead. When we skip the generator call at step $k+1$, we recover an approximate velocity from the cached endpoint and the current state and time:

$$\tilde{\mathbf{u}}_{k+1} = (\mathbf{x}_{t_{k+1}} - \hat{\mathbf{x}}_{0|t_k}) / t_{k+1}. \quad (8)$$

360	This is valid up to a local linearization around $\hat{\mathbf{x}}_{0 t_k}$:	408
361	because the clean endpoint is invariant along the de-	409
362	terministic part of the trajectory under linear flow,	
363	only the codebook innovation between steps k and	
364	$k+1$ moves the endpoint, and that movement is	
365	small relative to the cached value. Crucially, the full	
366	SDE/codebook correction in Eq. 1 is preserved at the	
367	skipped step; only the generator query is replaced.	
368	Schedule and warmup. We adopt the fixed alternat-	
369	ing schedule k is run $\Leftrightarrow k \equiv k_0 \pmod{2}$, where k_0 is the	
370	first SDE step in the current resolution stage. At every	
371	transition the cache is invalidated (the spatial resolu-	
372	tion and therefore the operating domain changes), and	
373	we additionally delay the first post-transition skip by	
374	one step (transition warmup): the cache is rebuilt at	
375	step $\tau_{s \rightarrow s+1} + 1$ before being reused at step $\tau_{s \rightarrow s+1} + 2$.	
376	The number of generator calls is unchanged relative to	
377	the no-warmup schedule; the only difference is which	
378	step's output drives the first post-transition skip.	
379	One-step radius. Empirically the cache is locally valid	
380	for one skipped step. We tested adaptive variants	
381	that allowed two or more consecutive skips when a	
382	running per-step error estimate fell below a thresh-	
383	old; quality collapsed as soon as the cap of two was	
384	actually triggered (Section 5). The mechanism is re-	
385	ursive cache staleness: the second skipped step plugs	
386	an already-extrapolated state back into the same lin-	
387	earization, and the error compounds. This radius-of-	
388	one observation rules out a large family of more ag-	
389	gressive schedules and identifies the fixed alternating	
390	pattern as Pareto-optimal under this caching scheme.	
391	3.4. GOP-level conditioning modes	
392	We retain the three GOP-conditioning modes of	
393	GVCC [23] and apply both acceleration mechanisms	
394	uniformly across them.	
395	T2V (intra-only). Each GOP is generated uncondi-	
396	tionally from text. The per-GOP bitstream consists	
397	of codebook bits only and the operating point is the	
398	lowest in our set (≤ 0.005 bpp at the bitrate hyper-	
399	parameters of Section 4). This is the analogue of an	
400	all-I-frame codec.	
401	I2V with autoregressive chaining. The first GOP	
402	takes the ground-truth first frame of the sequence as a	
403	“free” anchor; subsequent GOPs reuse the decoded last	
404	frame of the previous GOP as their anchor, avoiding re-	
405	peated transmission of boundary frames. To suppress	
406	drift in the high-noise tail latent frames, we apply a	
407	small quantized residual correction (8-bit, last latent	
	frame only) that the encoder transmits as a few hun-	408
	dred bytes per GOP.	409
	FLF2V with boundary sharing. Each GOP is con-	410
	ditioned on both its first and last frames. For a se-	411
	quence of N GOPs, we transmit $N+1$ boundary frames	412
	(compressed with CompressAI [1] at quality 4): the	413
	first GOP transmits two frames; each subsequent GOP	414
	reuses the previous GOP's last frame as its own first	415
	and transmits only one new last frame. This boundary-	416
	sharing scheme amortizes the boundary cost over long	417
	sequences and provides dual temporal anchors that re-	418
	duce per-frame variance within each GOP.	419
	The three modes occupy distinct points on the rate-	420
	quality plane: Section 4 reports them jointly.	421
	4. Experiments	422
	4.1. Setup	423
	Backbones. Unless otherwise noted, the video gen-	424
	erator is Wan 2.1-T2V-1.3B for T2V and Wan 2.1-	425
	FLF2V-14B-720P for FLF2V, matching the backbones	426
	reported in [23]. I2V uses Wan 2.1-I2V-14B-720P. All	427
	checkpoints are used in bf16 without any fine-tuning or	428
	distillation.	429
	Dataset. We evaluate on UVG-1080p [13] (seven	430
	sequences: Beauty, Bosphorus, HoneyBee, Jockey,	431
	ReadySteadyGo, ShakeNDry, YachtRide). Each se-	432
	quence is split into 33-frame GOPs with no overlap;	433
	sequences are processed at their native 1920×1080 res-	434
	olution and additionally at 1280×720 for cross-checks.	435
	The half-UVG evaluation in our main 1080p tables uses	436
	the first 9 GOPs (≈ 300 frames) per sequence, 63 GOPs	437
	total; we extend to full UVG (18 GOPs per sequence)	438
	in the camera-ready version.	439
	Hyperparameters. We follow the GVCC defaults: $T =$	440
	20 total sampling steps with $N=3$ deterministic ODE	441
	tail steps, codebook size $K=16384$, $M=64$ atoms per	442
	latent frame per step at 720p and $M=80$ at 1080p,	443
	SDE scale $g_{\text{scale}} = 3.0$. The multi-resolution schedule	444
	uses $S=3$ stages at 1080p ($480 \times 832 \rightarrow 720 \times 1280 \rightarrow$	445
	1080×1920) and $S=2$ stages at 720p ($480 \times 832 \rightarrow$	446
	720×1280), with transitions at codebook steps 6 and	447
	12 for $S=3$ and at step 8 for $S=2$. FLF2V uses	448
	CompressAI [1] quality 4 for boundary frames with $N+$	449
	1 boundary sharing.	450
	Metrics. We report PSNR (mean across all decoded	451
	frames vs. ground truth), MS-SSIM, and LPIPS	452
	(AlexNet backbone). Bitrate is reported as bits-per-	453
	pixel (BPP) computed over $H \times W \times F$ pixels per GOP.	454

Decoder wall-clock is the per-GOP time on a single NVIDIA RTX PRO 6000 Blackwell (96 GB) with bf16 inference and PyTorch 2.10, averaged across all GOPs of all sequences.

Compared methods. Traditional codecs HEVC [18] and VVC [2], learned codecs DCVC-FM [7] and DCVC-RT [5], generative codecs GLC-Video [15] and GNV-VD [12], zero-shot generative codec Free-GVC [9], and the published GVCC [23] are reported on UVG-1080p with numbers taken from the respective papers. We mark our entries GVCCTurbo-{T2V, I2V, FLF2V}; each entry uses the full acceleration stack (multi-resolution coding with the unified target, predictive skip with $p=2$ alternating schedule and transition warmup, and the DLC1 transition codebook). The codec backbone is otherwise identical to the published GVCC of the same conditioning mode, so any change in PSNR/LPIPS is attributable to the acceleration mechanisms and the bitrate of the transition codebook.

4.2. Main results: UVG 1080p

Table 1 summarizes our main RD-quality comparison.

FLF2V (Tier 2). GVCCTurbo-FLF2V at 1080p obtains 32.2 dB PSNR and 0.101 LPIPS at 0.0070 BPP, improving over the published GVCC-FLF2V operating point by +0.5 dB PSNR and -14% LPIPS at a 17% higher bitrate (the bitrate gap arises from boundary amortization at half UVG and is expected to close within $\sim 5\%$ at full UVG due to the $N+1$ boundary sharing scheme described in Section 3.4). Against all non-GVCC entries in Tier 2, our LPIPS is lowest by a clear margin (closest competitor: GNV-VD at 0.140); PSNR sits near GLC-Video’s 29.5 but with substantially better perceptual quality.

T2V (Tier 1). GVCCTurbo-T2V at 1080p (Wan-T2V-1.3B backbone) reaches 27.4 dB at 0.0030 BPP. This is 2.3 dB below the published GVCC-T2V operating point (Wan-T2V-14B at 29.7 dB) and reflects an unsurprising backbone gap: the 1.3B model is significantly more efficient to decode but loses spatial fidelity at 1080p relative to the 14B model. We retain this operating point in the table as our cheapest entry; the backbone scaling is an orthogonal axis that we did not optimize in this work.

I2V (Tier 3). GVCCTurbo-I2V at 1080p (Wan-I2V-14B backbone) reaches the same 32.2 dB / 0.087 LPIPS as the published GVCC-I2V at the same BPP. I2V

acceleration matches the published quality almost exactly: the tail- residual correction dominates the bit-stream and the codec-side compute savings transfer cleanly.

4.3. Decoder wall-clock

Table 2 reports per-GOP decoder wall-clock for the FLF2V-14B and T2V-1.3B backbones at 1080p. At 720p the T2V-1.3B pipeline reaches 19 s per GOP under the full acceleration stack ($S=2$, $T=20$ with $T_{\text{sde}}=17$ where half of the SDE steps reuse the cached endpoint). At 1080p the FLF2V-14B pipeline reaches 439 s per GOP with $S=3$ multi-resolution coding and predictive skip—approximately $1.6\times$ faster than the vanilla baseline at this resolution and backbone. The bulk of the remaining cost is the 1080p stage of the multi-resolution schedule itself; further reduction is possible by reducing the number of 1080p steps at a quality cost we quantify in Section 5.

The 720p T2V 19 s entry is the closest we get to real-time-class decoding (33 frames at 16 fps = 2.06 s of video reconstructed in 19 s, i.e. $\sim 9\times$ slower than playback). The 1080p FLF2V entry, while substantially slower, remains $\sim 2\times$ faster than the corresponding vanilla GVCC operating point at the same reconstruction quality.

5. Ablations

We isolate the contribution of each acceleration mechanism on full UVG 720p with the Wan 2.1-T2V-1.3B backbone, $T=20$, $M=64$, $K=16384$, $g_{\text{scale}}=3.0$ unless otherwise noted. Decode time is reported per GOP.

5.1. The target-domain mismatch and its fix

Table 3 compares the naive multi-resolution baseline (low-stage target via Eq. 3) against the unified target (Eq. 5, our fix) at $S=2$ on UVG-720p, together with a joint-oracle ceiling that uses the encoder’s ground-truth high-band content at the transition step (a non-deployable upper bound that bounds how much the codebook-driven scheme can recover).

Three observations follow from Table 3.

The ceiling is structural under the naive target. The naive multi-resolution scheme suffers a 1.44 dB PSNR loss versus the single-resolution baseline. Adding the DLC1 transition codebook recovers half of it (-0.65 vs. -1.44), and even the joint oracle—which transmits no bits but assumes a perfect high-band content at transition—only reaches -1.26 . The 1.26 dB gap to the single-resolution baseline is therefore not a code-

Table 1. Comparison on UVG 1080p across three representative bitrate regimes. Format follows [23] (their Table 2): each GVCC variant is reported at its native operating point. Baseline LPIPS values are the published readings (AlexNet backbone for ours; baseline backbone may differ). Bold: lowest LPIPS per tier. GVCCTurbo rows use the full acceleration stack of Section 3.

Method	Tier 1: ~ 0.003 bpp			Tier 2: ~ 0.006 bpp			Tier 3: ~ 0.05 bpp		
	PSNR $\uparrow$	LPIPS $\downarrow$	BPP	PSNR $\uparrow$	LPIPS $\downarrow$	BPP	PSNR $\uparrow$	LPIPS $\downarrow$	BPP
Traditional									
HEVC [18]	29.8	0.385	0.003	30.2	0.360	0.006	34.7	0.115	0.050
VVC [2]	31.5	0.325	0.003	31.8	0.315	0.006	36.0	0.112	0.050
Learned									
DCVC-FM [7]	34.0	0.286	0.003	34.7	0.265	0.006	37.0	0.110	0.050
DCVC-RT [5]	34.1	0.285	0.003	34.6	0.261	0.006	39.7	0.115	0.050
Generative (trained)									
GLC-Video [15]	27.5	0.160	0.003	29.5	0.145	0.006	32.0	0.109	0.050
GNVC-VD [12]	26.8	0.165	0.003	30.0	0.140	0.006	—	—	—
Generative (zero-shot)									
Free-GVC [9]	—	0.185	0.003	—	0.155	0.006	—	0.105	0.050
GVCC-T2V [23]	29.7	0.133	0.0027	31.7	0.117	0.006	32.2	0.087	0.050
GVCC-FLF2V [23]									
GVCC-I2V [23]									
GVCCTurbo-T2V	27.4	0.214	0.0030	32.2	0.101	0.0070	32.2	0.087	0.050
GVCCTurbo-FLF2V									
GVCCTurbo-I2V									

Table 2. Per-GOP decoder wall-clock at 1080p on a single RTX PRO 6000 (96 GB, bf16). All methods use the same backbone per row. Vanilla denotes the GVCC pipeline without any of our acceleration mechanisms; +vec adds the vectorized codebook RNG (an implementation fix); +x0c adds predictive skip with $p = 2$ alternating schedule and transition warmup; +MR adds the multi-resolution coding schedule. The final row is the full GVCCTurbo stack.

Method	Backbone	Time/GOP	Speedup
Vanilla	FLF2V-14B	~ 700 s	1.0 $\times$
+vec	FLF2V-14B	~ 425 s	1.6 $\times$
+vec +x0c	FLF2V-14B	~ 247 s	2.8 $\times$
+vec +x0c +MR	FLF2V-14B	439s [†]	1.6 $\times$
Vanilla	T2V-1.3B	~ 130 s	1.0 $\times$
+vec +x0c	T2V-1.3B	19s	6.8 $\times$ (720p)
+vec +x0c +MR	T2V-1.3B	108s	1.2 $\times$ (1080p)

[†] MR at 1080p uses $S = 3$ ($480 \rightarrow 720 \rightarrow 1080$), which adds compute at the 1080p stage. The MR speedup is more pronounced at 720p with $S = 2$.

Table 3. Multi-resolution coding with naive vs. unified target. Full UVG $\times 3$ GOPs at 720p with Wan 2.1-T2V-1.3B, $S = 2$ schedule ($480 \rightarrow 720$). Δ is relative to the single-resolution baseline (PSNR 28.72, BPP 0.00483).

Target	PSNR (dB)	BPP	Δ PSNR
Single-resolution baseline	28.72	0.00483	—
MR naive (Eq. 3)	27.28	0.00481	-1.44
MR naive + DLC1 transition	28.07	0.00481	-0.65
MR naive + joint oracle	28.89	0.00481	-1.26
MR unified (ours)	28.50	0.00481	-0.22
MR unified + DLC1 transition	28.50	0.00511	-0.22
MR unified + joint oracle	28.62	0.00453	-0.10

of the naive target before use, intending to “soften” the domain gap. Both hurt: by ≈ 0.5 dB on Bosphorus and HoneyBee relative to the raw unified target. Codebook-matching is spectrally sensitive, and rescaling re-introduces the domain mismatch in a more subtle form.

The fix is essentially one line of code. Replacing Eq. 3 with Eq. 5 collapses the gap to -0.22 dB at the same bitrate. The joint-oracle ceiling under the unified target is -0.10 dB, so the DLC1 transition codebook captures roughly 95% of the remaining recoverable information

book expressiveness problem; it is fundamental to the codec target.

Magnitude alignment is not the fix. We tried two natural variants of the unified target that rescale $\mathcal{T}_L(\text{VAE}(\mathbf{V}))$ to match the per-channel std (or norm)

Table 4. Caching choice for the per-step generator query, $p=2$ alternating schedule, full UVG 720p, multi-resolution $S=2$. Time is per-GOP decoder wall-clock; Δ PSNR is relative to no caching.

Cache	PSNR (dB)	Time/GOP	Δ PSNR	S	τ	PSNR (dB)	BPP
None (full T generator calls)	28.50	25s	Single-resolution	1	—	30.15	0.00483
Velocity cache (Eq. 6)	26.71	19s	480 $\rightarrow$ 720	2	{8}	28.72	0.00481
Clean-endpoint cache (ours, Eq. 8)	28.45	19s	480 $\rightarrow$ 720 (early τ)	2	{6}	28.51	0.00481
			480 $\rightarrow$ 720 (late τ)	2	{10}	28.65	0.00481
			272 $\rightarrow$ 480 $\rightarrow$ 720	3	{4, 12}	27.76	0.00506
			272 $\rightarrow$ 480 $\rightarrow$ 720 (oracle)	3	{4, 12}	28.07	0.00451

Table 5. Adaptive predictive skip with a per-stage cap on consecutive skips. HoneyBee, single GOP, 720p, Wan-T2V-1.3B. Threshold tuned to keep average skip rate near 50%.

Schedule	PSNR (dB)	Time/GOP	Our robustness mode
Fixed alternating ($p=2$, cap $\equiv 1$)	29.47	18s	large, never skip when the running per-step error is too large, never schedule a faster pattern; quality therefore matches the fixed alternating schedule but speed is bounded above by the same.
Adaptive, cap = 1, thr 0.30	29.48	19s	our robustness mode
Adaptive, cap = 2, thr 0.15 (triggered)	29.08	17s	cap $\equiv 1$ is a robustness mechanism, not a new acceleration frontier. The fixed alternating schedule is Pareto-optimal under the clean-endpoint caching scheme.
Adaptive, no cap, thr 0.40 (triggered)	25.95	13s	

at $< 1\%$ bitrate overhead.

5.2. Predictive skip: caching choice and schedule

Table 4 compares no caching, velocity caching, and clean-endpoint caching on UVG-720p with Wan 2.1-T2V-1.3B, at the fixed alternating $p=2$ schedule (every other SDE step skipped) with transition warmup after the multi-resolution lift.

The clean-endpoint cache reduces decoder time by $1.32\times$ at a 0.05 dB PSNR cost; the same skip rate with naive velocity caching costs 1.79 dB—a $36\times$ larger PSNR drop. The two schemes are otherwise identical (same skip schedule, same number of generator calls); the difference comes entirely from what is cached. The velocity cache is computed at a stale state and the trajectory has moved away from $\mathbf{x}_{t_k}$ by both the deterministic drift and the codebook innovation between steps k and $k+1$. The endpoint cache is geometrically invariant along the deterministic part of the trajectory, so only the codebook-induced shift between steps remains as approximation error.

The one-step radius. Table 5 ablates the maximum allowed number of consecutive skips, holding the average skip rate fixed via an adaptive threshold (we transmit a 1-bit-per-SDE-step skip mask in the bitstream when adaptivity is on).

Whenever the cap of 2 is actually triggered (i.e., the adaptive controller picks two consecutive skips), quality drops sharply. Removing the cap entirely allows long skip runs and quality collapses by 3.5 dB. With a hard cap of 1, the adaptive controller can only cancel

Table 6. Number of multi-resolution stages and transition placement. τ lists the SDE step indices at which each transition occurs. Δ PSNR relative to single-resolution baseline (30.15 dB).

Cache	PSNR (dB)	Time/GOP	Δ PSNR	S	τ	PSNR (dB)	BPP
None (full T generator calls)	28.50	25s	Single-resolution	1	—	30.15	0.00483
Velocity cache (Eq. 6)	26.71	19s	480 $\rightarrow$ 720	2	{8}	28.72	0.00481
Clean-endpoint cache (ours, Eq. 8)	28.45	19s	480 $\rightarrow$ 720 (early τ)	2	{6}	28.51	0.00481
			480 $\rightarrow$ 720 (late τ)	2	{10}	28.65	0.00481
			272 $\rightarrow$ 480 $\rightarrow$ 720	3	{4, 12}	27.76	0.00506
			272 $\rightarrow$ 480 $\rightarrow$ 720 (oracle)	3	{4, 12}	28.07	0.00451

large, never skip when the running per-step error is too large, never schedule a faster pattern; quality therefore matches the fixed alternating schedule but speed is bounded above by the same. Adaptive caching with cap $\equiv 1$ is a robustness mechanism, not a new acceleration frontier. The fixed alternating schedule is Pareto-optimal under the clean-endpoint caching scheme.

5.3. Stage count and transition placement

Table 6 ablates the number of multi-resolution stages S and the placement of transition steps on UVG-720p (Bosphorus + HoneyBee, 3 GOPs each, Wan-T2V-1.3B). All entries use the unified target and DLC1 transition codebook.

$S=2$ is the sweet spot on Wan 2.1. Two stages with a transition at the midpoint dominate $S=3$ schedules that include a 272p floor. The drop at $S=3$ is not a coding artifact: even the joint oracle at $S=3$ (28.07 dB) cannot reach the $S=2$ codebook result (28.72 dB). The cause is that Wan 2.1 was not trained at latents below the 480p band; running the first few denoising steps on an even smaller latent operates the generator out-of-distribution, and the spectrally lifted state at transition is no longer a good initialization for the higher-resolution stage. Moving the low floor up to 480p eliminates the problem; further increasing S within [480, 720] adds transition overhead without quality benefit.

Transition timing. At $S=2$, sweeping the transition step in {6, 8, 10} yields 28.51/28.72/28.65 dB respectively. The step-8 choice (approximately midway through the SDE phase) is best by a small margin (+0.21 over step-6). This matches the spectral diffusion intuition [21]: too early a switch deprives the low stage of useful denoising, too late a switch wastes generator capacity at high resolution.

5.4. Vectorized codebook RNG: implementation, not algorithm

The Turbo-DDCM [19] codebook construction generates K atoms per step per latent frame by independent $\text{randn}(D)$ calls. In our implementation, replacing these with a single batched $\text{randn}(K, D)$ per step reduces decoder wall-clock by $\approx 1.9\times$ at identical bitstream content (seed-consistent on both sides). This is an engineering fix—it does not appear in the bitstream—but we report it for transparency: all timing numbers in this paper, including the published GVCC baseline rows in Table 2, include this fix unless explicitly marked vanilla.

6. Conclusion

We have presented GVCCTurbo, a zero-shot generative video codec that retains the codebook-driven backbone of GVCC but reduces per-GOP decode time by up to $1.9\times$ on Wan 2.1 backbones at preserved quality. The two mechanisms responsible for the speedup—unifying multi-resolution stages in a single latent domain, and caching the trajectory endpoint rather than the velocity for predictive skipping—are simple in implementation and grounded in trajectory geometry. Both required only small code-level changes; neither requires retraining or any change to the bitstream format beyond an optional sub-percent overhead for the DLC1 transition codebook.

The acceleration mechanisms transfer cleanly to the FLF2V mode: at 1080p with boundary-sharing dual-anchor conditioning, GVCCTurbo matches the published GVCC operating point in bitrate while improving PSNR by +0.5 dB and LPIPS by -14% at approximately $2\times$ the decode speed. They are partial in the T2V mode at the chosen lightweight backbone (Wan 2.1-1.3B), where the dominant residual is the backbone’s limited capacity at 1080p rather than the codec’s compute budget.

Limitations and future work. The decoder is still substantially slower than learned codecs of the DCVC family ($\sim 10^2$ vs. $\sim 10^{-1}$ s/GOP), and the main remaining cost is the per-generator-call latency of the high-resolution stage. Two directions appear most promising. First, the predictive-skip mechanism’s one-step-radius observation rules out simple multi-step caching, but does not preclude a learned step-distillation that retrains a few-step generator compatible with the encoder’s per-step reproducibility requirement—this is an open research direction with no clean fix today. Second, a lightweight generator natively trained at 1080p (the closest existing candidate at the time of writing,

LTX-Video [8], is a strong starting point) would change the backbone-vs-codec balance of the cost; we plan to investigate this trade in future work.

We release code, full per-GOP measurements, and codebook bitstreams to support reproducibility and further investigation along the multi-resolution and predictive-skip axes.

References

- [1] Jean Bégaint, Fabien Racapé, Simon Feltman, and Akshay Pushparaja. CompressAI: a PyTorch library and evaluation platform for end-to-end compression research. In arXiv preprint arXiv:2011.03029, 2020. 5
- [2] Benjamin Bross, Ye-Kui Wang, Yan Ye, Shan Liu, Jianle Chen, Gary J. Sullivan, and Jens-Rainer Ohm. Overview of the versatile video coding (VVC) standard and its applications. IEEE Transactions on Circuits and Systems for Video Technology, 31(10):3736–3764, 2021. 2, 6, 7
- [3] Patrick Esser et al. Scaling rectified flow transformers for high-resolution image synthesis. In International Conference on Machine Learning (ICML), 2024. 3
- [4] Jonathan Ho, Chitwan Saharia, William Chan, David J. Fleet, Mohammad Norouzi, and Tim Salimans. Cascaded diffusion models for high fidelity image generation. In Journal of Machine Learning Research, 2022. 3
- [5] Zhihao Jia, Bin Li, Houqiang Li, and Yan Lu. DCVC-RT: Real-time neural video compression with feature modulation. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025. 1, 2, 6, 7
- [6] J. Kim et al. Inference-time acceleration of diffusion models via velocity caching. arXiv preprint, 2025. 3
- [7] Jiahao Li, Bin Li, and Yan Lu. Neural video compression with feature modulation. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024. DCVC-FM. 1, 2, 6, 7
- [8] Lightricks. LTX-Video: Realtime video latent diffusion, 2025. arXiv:2501.00103. 9
- [9] Yi Ling et al. Free-GVC: Training-free generative video compression via reverse channel coding on pretrained diffusion models. arXiv preprint, 2026. 2, 6, 7
- [10] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. In arXiv preprint, 2023. 3
- [11] Xinyin Ma, Gongfan Fang, and Xinchao Wang. Deep-Cache: Accelerating diffusion models for free. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024. 3
- [12] Hao Mao et al. GNVC-VD: Generative neural video compression with visual diffusion. arXiv preprint, 2025. 2, 6, 7
- [13] Alexandre Mercat, Marko Viitanen, and Jarno Vanne. UVG dataset: 50/120fps 4K sequences for video codec

- analysis and development. In ACM Multimedia Systems Conference, 2020. 5
- [14] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. SDXL: Improving latent diffusion models for high-resolution image synthesis. In International Conference on Learning Representations (ICLR), 2024. 3
- [15] Yifan Qi et al. GLC-Video: Generative latent compression for video at ultra-low bitrates. arXiv preprint, 2025. 2, 6, 7
- [16] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In International Conference on Learning Representations (ICLR), 2022. 3
- [17] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In International Conference on Machine Learning (ICML), 2023. 3
- [18] Gary J. Sullivan, Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. Overview of the high efficiency video coding (HEVC) standard. IEEE Transactions on Circuits and Systems for Video Technology, 22(12): 1649–1668, 2012. 2, 6, 7
- [19] Yair Vaisman et al. Turbo-DDCM: Fast and accurate single-image compression with multi-atom codebooks. In Advances in Neural Information Processing Systems, 2025. 3, 9
- [20] Wan-AI Team. Wan: Open and advanced large-scale video generative models, 2025. Wan 2.1, Alibaba. 2, 3
- [21] Tianbo Xiao et al. Spectral progressive diffusion. In arXiv preprint arXiv:2605.18736, 2026. 2, 3, 8
- [22] Enze Xie, Junsong Chen, Junyu Chen, Han Cai, et al. SANA: Efficient high-resolution image synthesis with linear diffusion transformer. In arXiv preprint, 2024. 3
- [23] Ziyue Zeng, Xun Su, Haoyuan Liu, Bingyu Lu, Yui Tatsumi, and Hiroshi Watanabe. GVCC: Zero-shot video compression via codebook-driven stochastic rectified flow. arXiv preprint arXiv:2603.26571, 2026. 1, 2, 3, 5, 6, 7